

```

1 import math
2 import random
3 SCAN_POINTS = [(0, 6), (6, 0), (0, -6), (-6, 0)]
4
5 def level(point, source, spread):
6     return 100 / (1 + (math.hypot(point[0] - source[0], point[1] - source[1]) / spread) ** 2)
7
8 def find_best_scan_point(choices, source, spread):
9     best_point = choices[0]
10    best_score = level(choices[0], source, spread)
11
12    for point in choices:
13        score = level(point, source, spread)
14        if score > best_score:
15            best_score = score
16            best_point = point
17
18    return best_point, best_score
19
20 def move_toward_source(start, source, spread, steps):
21    path = [start]
22    current = start
23
24    for _ in range(steps):
25        x, y = current
26        choices = [(x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)]
27        next_point, next_score = find_best_scan_point(choices, source, spread)
28        if next_score <= level(current, source, spread):
29            break
30
31        current = next_point
32        path.append(current)
33    return current, path
34
35 def main():
36    print("Pollution Source Search Program")
37    print("This project models how robot spiders use pollution readings to locate a hidden pollution source near their mother nest.")
38    print("Input explanation: enter one integer difficulty value.")
39    print("A larger difficulty means the source is farther away and harder to find.")
40    print()
41
42    while True:
43        try:
44            difficulty = int(input("Enter a difficulty (1 or higher, suggested: 3): "))
45            if difficulty >= 1:
46                break
47        except EOFError:
48            print("\nNo input was detected, so the program will use the suggested value 3.")
49            difficulty = 3
50            break
51        except ValueError:
52            pass
53        print("Invalid input. Please enter an integer that is 1 or higher.")
54
55    radius = 6 + difficulty * 2
56    spread = max(2.0, 8.0 - difficulty * 0.5)
57    angle = random.uniform(0, 2 * math.pi)
58    source = (round(radius * math.cos(angle)), round(radius * math.sin(angle)))
59
60    start_point, start_score = find_best_scan_point(SCAN_POINTS, source, spread)
61    current, path = move_toward_source(start_point, source, spread, difficulty + 3)
62
63    print()
64    print(
65        f"At difficulty {difficulty}, the pollution source is placed about {radius} units from the mother nest "
66        f"With a spread of {spread:.1f}. The program first checks the scan points {SCAN_POINTS}, "
67        f"selects {start_point} as the strongest starting point with a reading of {start_score:.2f}, "
68        f"and then follows stronger readings along {path}. Based on that search, the program predicts "
69        f"the source at {current}, while the actual source is {source}."
70    )
71
72 if __name__ == "__main__":
73     main()
74

```